Light

Computational Thinking

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Algorithms
2. Decomposition and abstraction
3. Representing algorithms
4. Trace tables
Worked examples
Quick reference
Typical pitfalls

AQA GCSE Computer Science — **Computational Thinking (8525 Paper 1)**. This topic covers algorithms, decomposition, abstraction, representing algorithms using pseudo-code and flowcharts, identifying inputs/processing/outputs, using trace tables, and comparing algorithm efficiency.

Exam questions typically ask you to **define** key terms, **explain** decomposition and abstraction, **complete** trace tables, **write** algorithms in pseudo-code or program code, **draw** flowcharts, and **compare** the efficiency of algorithms. This is Paper 1 content.

Why computational thinking matters

Computational thinking is a problem-solving approach used by computer scientists. It involves breaking down complex problems into smaller parts, identifying patterns, removing unnecessary detail, and creating step-by-step solutions that a computer can follow.

These skills are essential for programming — before you can write code, you need to understand the problem and design a solution. Computational thinking helps you

approach any problem in a logical, systematic way.

- **Decomposition:** Breaking a problem into smaller, manageable parts
- **Abstraction:** Focusing on important information and ignoring unnecessary detail
- **Algorithmic thinking:** Creating step-by-step solutions

What you must know

This checklist covers the key exam-visible concepts from the AQA specification. You should be able to explain each point clearly.

- **Define** the term algorithm.
- **Understand** that a computer program is an implementation of an algorithm.
- **Understand** that an algorithm is not the same as a computer program.
- **Define** and **explain** the term decomposition.
- **Define** and **explain** the term abstraction.
- **Use** a systematic approach to problem solving and algorithm creation.
- **Represent** algorithms using pseudo-code, program code, and flowcharts.
- **Explain** algorithms in terms of their inputs, processing, and outputs.
- **Use** trace tables to determine how algorithms work.
- **Determine** the purpose of simple algorithms.
- **Understand** that more than one algorithm can solve the same problem.
- **Compare** the efficiency of algorithms (time efficiency only).

> **Specification note.** AQA 8525 uses its own pseudo-code standard — exam questions will use this format. You can answer in any suitable pseudo-code format, but you must be able to read and understand the AQA version. Formal comparisons of algorithmic efficiency (such as Big-O notation) are not required — only informal time efficiency comparisons.

Toolkit

Key definitions

Term	Definition (exam answer)
Algorithm	A sequence of steps that can be followed to complete a task
Decomposition	Breaking a problem into a number of sub-problems, so that each sub-problem accomplishes an identifiable task
Abstraction	The process of removing unnecessary detail from a problem
Trace table	A table used to trace the values of variables as an algorithm is executed

Flowchart symbols

Symbol	Shape	Purpose
Terminal	Oval / Rounded rectangle	Start or End of the algorithm
Process	Rectangle	An action or calculation
Decision	Diamond	A Yes/No question (condition)
Input/Output	Parallelogram	Data entering or leaving the algorithm
Arrow	Line with arrowhead	Flow of control (direction)

Inputs, processing, and outputs

Inputs
Data entering the algorithm

Processing
Actions performed on data

Outputs
Results produced

1. Understanding algorithms

An **algorithm** is the foundation of all computer programs. Before you can write code, you need an algorithm that describes the solution to a problem.

1.1 What is an algorithm?

Definition: An algorithm is a sequence of steps that can be followed to complete a task.

Key points about algorithms:

- **Sequence of steps:** An algorithm is a set of instructions in a specific order
- **Can be followed:** The steps must be clear and unambiguous
- **Completes a task:** An algorithm has a purpose — it solves a problem or achieves a goal
- **Finite:** An algorithm must eventually finish (it shouldn't run forever)

Algorithms exist in everyday life, not just in computing. Examples include:

- A recipe for baking a cake
- Instructions for assembling furniture
- Directions to navigate from one place to another
- The steps to solve a mathematical equation

1.2 Algorithms vs programs

It is important to understand the difference between an algorithm and a computer program:

Algorithm

- A sequence of steps to solve a problem
- Can be written in any format (words, diagrams, pseudo-code)
- Language-independent
- Describes WHAT to do

Computer program

- An implementation of an algorithm
- Written in a specific programming language
- Can be executed by a computer
- Describes HOW to do it in code

> **Key distinction:** A computer program is an implementation of an algorithm. An algorithm is NOT a computer program — it is the plan or design that the program follows. The same algorithm can be implemented in many different programming languages.

1.3 Inputs, processing, and outputs

Every algorithm can be explained in terms of three components:

- **Inputs:** The data that enters the algorithm — what information does it need to start?
- **Processing:** The actions performed on the data — what calculations, comparisons, or transformations happen?
- **Outputs:** The results produced by the algorithm — what information does it produce?

Example: Calculate the area of a rectangle

- **Inputs:** Length and width of the rectangle
- **Processing:** Multiply length by width
- **Outputs:** The area of the rectangle

In exam questions, you may be asked to identify the inputs, processing, and outputs within an algorithm. Look for:

- **Inputs:** Variables that receive data from the user or from external sources (e.g., `INPUT name`)
- **Processing:** Calculations, comparisons, loops, and conditional statements
- **Outputs:** Data displayed to the user or returned by the algorithm (e.g., `OUTPUT result`)

1.4 Algorithm efficiency

More than one algorithm can solve the same problem. However, some algorithms are more **efficient** than others. At GCSE level, efficiency refers to **time efficiency** — how quickly an algorithm completes its task.

Factors that affect time efficiency:

- **Number of operations:** Algorithms that perform fewer operations are generally faster
- **Data size:** Some algorithms slow down dramatically as the amount of data increases
- **Algorithm design:** A well-designed algorithm can be much faster than a poorly designed one

Example: Finding a name in a list

Linear search: Check each name one by one from the start. If the list has 1000 names and the target is last, you check all 1000.

Binary search: If the list is sorted, repeatedly halve the search area. For 1000 names, you need at most about 10 checks.

Binary search is much more efficient for large sorted lists, but requires the data to be sorted first.

Worked Example 1 – Defining an algorithm

Question: Define the term algorithm. [1 mark]

Model answer:
A sequence of steps that can be followed to complete a task **(1)**.

Exam tip: Learn this definition exactly. Key elements: "sequence of steps", "can be followed", "complete a task".

Worked Example 2 – Algorithm vs program

Question: Explain the difference between an algorithm and a computer program. [2 marks]

Model answer:
An algorithm is a sequence of steps to solve a problem and can be written in any format **(1)**. A computer program is an implementation of an algorithm written in a specific programming language that can be executed by a computer **(1)**.

Exam tip: The key point is that an algorithm is the PLAN, and a program is the IMPLEMENTATION. Same algorithm, different languages = different programs.

Worked Example 3 – Inputs, processing, outputs

Question: The following algorithm calculates the average of three numbers:

```
INPUT num1 INPUT num2 INPUT num3 total ← num1 + num2 + num3 average ←
total / 3 OUTPUT average
```

Identify the inputs, processing, and outputs in this algorithm. [3 marks]

Model answer:
Inputs: num1, num2, num3 **(1)**
Processing: Adding the three numbers together, then dividing by 3 **(1)**
Outputs: average **(1)**

Exam tip: INPUT statements show inputs. Calculations and operations are processing. OUTPUT statements show outputs.

Worked Example 4 – Algorithm efficiency

Question: Two algorithms both find the largest number in a list. Algorithm A compares every pair of numbers. Algorithm B goes through the list once, keeping track of the largest seen so far. Which algorithm is more efficient and why? [2 marks]

Model answer:
Algorithm B is more efficient **(1)** because it only needs to go through the list once, performing fewer comparisons than Algorithm A which compares every pair **(1)**.

Exam tip: Efficiency questions focus on TIME — which algorithm takes fewer steps or less time to complete?

2. Decomposition and abstraction

Decomposition and **abstraction** are two key computational thinking techniques. They help programmers break down complex problems into manageable parts and focus on what's important.

2.1 Decomposition

Definition: Decomposition means breaking a problem into a number of sub-problems, so that each sub-problem accomplishes an identifiable task, which might itself be further subdivided.

Decomposition makes complex problems easier to solve by:

- **Making problems manageable:** Smaller problems are easier to understand and solve

- **Allowing teamwork:** Different people can work on different sub-problems
- **Enabling code reuse:** Solutions to sub-problems can be reused elsewhere
- **Making testing easier:** Each sub-problem can be tested independently

Example: Creating a game

The problem "create a computer game" could be decomposed into:

- Design the user interface
- Create player movement
- Create enemy behaviour
- Implement scoring system
- Add sound effects
- Create game over screen

Each of these sub-problems can be further decomposed. For example, "create player movement" might include: move left, move right, jump, crouch.

2.2 Abstraction

Definition: Abstraction is the process of removing unnecessary detail from a problem.

Abstraction helps by:

- **Focusing on what matters:** Ignore details that don't affect the solution
- **Simplifying the problem:** A simpler model is easier to work with
- **Hiding complexity:** Users don't need to understand how everything works internally
- **Creating general solutions:** Abstract solutions can apply to many similar problems

Example: London Underground map

The London Underground map is a famous example of abstraction:

- **Included:** Station names, lines, connections between stations
- **Removed:** Actual geographical distances, exact locations, street-level detail

The map removes unnecessary detail (exact distances) and focuses on what passengers need (which stations connect to which).

Example: Driving a car

When you drive a car, you use abstraction:

- **What you need to know:** Steering wheel turns the car, pedals control speed
- **What you don't need to know:** How the engine combustion works, how the gearbox mechanism functions internally

The car's controls are an abstraction — they hide the complex internal details.

2.3 Decomposition vs abstraction

These two techniques work together but serve different purposes:

Decomposition	Abstraction
Breaks a problem into smaller parts	Removes unnecessary detail from a problem

Decomposition	Abstraction
Makes complex problems manageable	Simplifies problems by focusing on what matters
Creates sub-problems that can be solved independently	Creates a simplified model of the problem
Example: Splitting a game into player, enemies, scoring	Example: Representing a player as just position and health, not every detail

Worked Example 5 – Defining decomposition

Question: Define the term decomposition. [2 marks]

Model answer:
Decomposition means breaking a problem into a number of sub-problems **(1)**, so that each sub-problem accomplishes an identifiable task **(1)**.

Exam tip: Both parts are needed for full marks: "breaking into sub-problems" AND "each accomplishes an identifiable task".

Worked Example 6 – Applying decomposition

Question: A programmer is creating a quiz application. Decompose this problem into four sub-problems. [4 marks]

Model answer:
1. Display questions to the user **(1)**
2. Accept and validate user answers **(1)**
3. Check if answers are correct **(1)**
4. Calculate and display the final score **(1)**

Exam tip: Each sub-problem should be a distinct, identifiable task. Make sure they don't overlap and together they cover the whole problem.

Worked Example 7 – Defining abstraction

Question: Define the term abstraction. [1 mark]

Model answer:

Abstraction is the process of removing unnecessary detail from a problem **(1)**.

Exam tip: The key phrase is "removing unnecessary detail". You could also say "focusing on essential information" or "hiding complexity".

Worked Example 8 – Applying abstraction

Question: A programmer is creating a simulation of a school. Explain one example of abstraction they might use. [2 marks]

Model answer:
The programmer might represent each student using only their name and year group **(1)**, removing unnecessary details such as their home address, shoe size, or favourite colour **(1)**.

Exam tip: Explain WHAT is kept and WHAT is removed. The removed details should genuinely be unnecessary for the purpose.

3. Representing algorithms

Algorithms can be represented in several ways. AQA requires you to understand and use three main representations: **pseudo-code**, **program code**, and **flowcharts**.

3.1 Pseudo-code

Pseudo-code is a way of writing algorithms using a structured, code-like format that is not tied to any specific programming language. It uses plain English mixed with programming-style keywords.

Advantages of pseudo-code:

- **Language-independent:** Anyone can read it, regardless of which programming language they know
- **Easy to write:** No need to worry about exact syntax rules
- **Focuses on logic:** Concentrates on the algorithm, not the programming details

AQA pseudo-code conventions

AQA uses these standard pseudo-code conventions:

Operation	AQA Pseudo-code
Assignment	`variable ← value`
Input	`INPUT variable` or `variable ← USERINPUT`
Output	`OUTPUT value`
Selection	`IF...THEN...ELSE...ENDIF`

Operation	AQA Pseudo-code
Loop (count)	`FOR...TO...ENDFOR`
Loop (condition)	`WHILE...ENDWHILE` or `REPEAT...UNTIL`

```
Example: Check if a number is positive, negative, or zero INPUT number IF
number > 0 THEN OUTPUT "Positive" ELSE IF number < 0 THEN OUTPUT
"Negative" ELSE OUTPUT "Zero" ENDIF
```

3.2 Program code (Python)

Program code is an algorithm written in a specific programming language. AQA supports Python 3, VB.NET, and C#. Here we use Python as it is the most common choice.

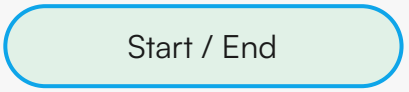
```python
# Example: Check if a number is positive, negative, or zero number =
int(input("Enter a number: ")) if number > 0: print("Positive") elif
number < 0: print("Negative") else: print("Zero")
```

3.3 Flowcharts

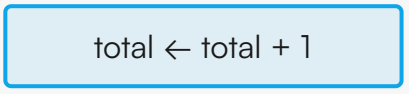
A **flowchart** is a visual representation of an algorithm using standard symbols connected by arrows.

Flowchart symbols

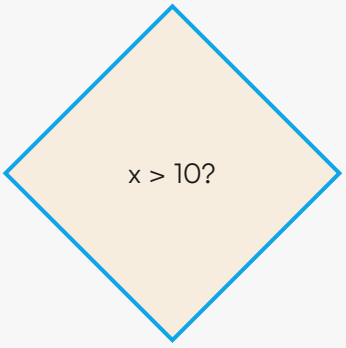
Terminal (Start/End): Oval shape



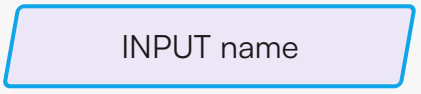
Process: Rectangle



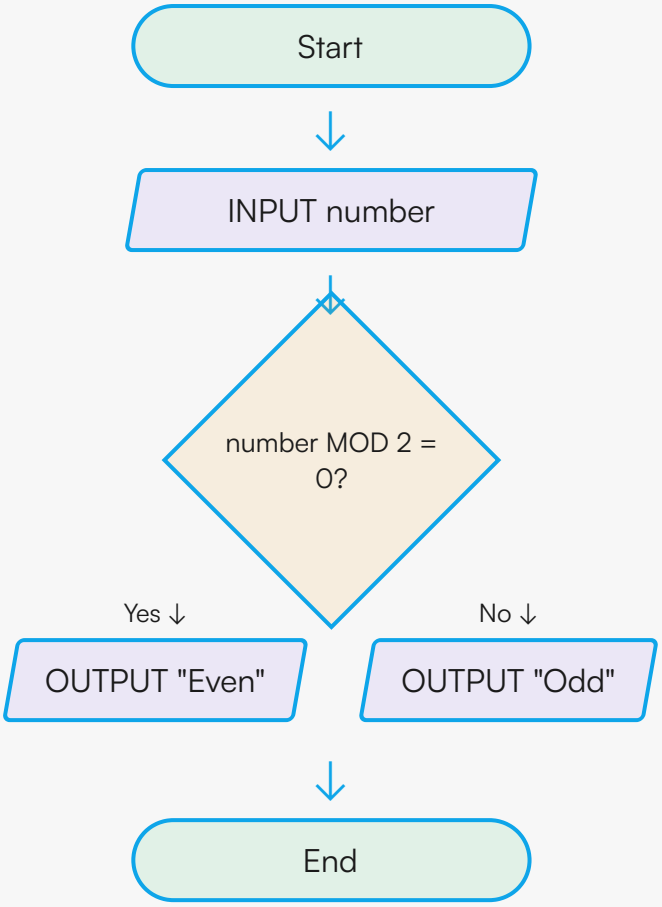
Decision: Diamond



Input/Output: Parallelogram



Example flowchart: Is a number even or odd?



3.4 Choosing a representation

In exams, the question will specify which representation to use. Each has advantages:

Representation	Best for
Pseudo-code	Writing algorithms quickly; planning before coding; explaining logic
Program code	Implementing solutions that can actually run; testing algorithms
Flowcharts	Visualising the flow of an algorithm; showing decision points; presenting to non-programmers

Worked Example 9 — Writing pseudo-code

Question: Write a pseudo-code algorithm that inputs a temperature in Celsius and outputs whether it is freezing (below 0), cold (0-15), warm (16-25), or hot (above 25). [4 marks]

Model answer:

```
INPUT temperature IF temperature < 0 THEN OUTPUT "Freezing" ELSE
IF temperature <= 15 THEN OUTPUT "Cold" ELSE IF temperature <= 25
THEN OUTPUT "Warm" ELSE OUTPUT "Hot" ENDIF
```

Exam tip: Check boundary conditions carefully — the question says 0-15 is cold, so temperature = 0 should output "Cold", not "Freezing".

Worked Example 10 – Writing Python code

Question: Write a Python program that asks the user to enter 5 numbers and outputs the total. [4 marks]

Model answer:

```
total = 0 for i in range(5): number = int(input("Enter a number:
")) total = total + number print(total)
```

Exam tip: Use meaningful variable names, correct syntax, and appropriate data types (int for whole numbers).

4. Trace tables

A **trace table** is a technique for working through an algorithm step by step, recording the values of variables at each stage. Trace tables help you understand how an algorithm works and identify its purpose.

4.1 What is a trace table?

A trace table has:

- **Columns:** One for each variable in the algorithm, plus optionally a column for outputs
- **Rows:** One for each step where a variable changes value

To complete a trace table:

1. Identify all the variables in the algorithm
2. Create a column for each variable
3. Work through the algorithm line by line
4. Each time a variable changes, add a new row with the updated value
5. Continue until the algorithm finishes

4.2 Example trace table

Consider this algorithm:

```
x ← 1 y ← 5 WHILE x < y x ← x + 1 y ← y - 1 ENDWHILE OUTPUT x
```

The trace table for this algorithm:

x	y	x < y?	Output
1	5	True	
2	4	True	
3	3	False	3

The algorithm outputs **3**. Its purpose is to find the midpoint between the initial values of x and y.

4.3 Tips for trace tables

- **Include all variables:** Don't forget counters and temporary variables
- **Track loop iterations:** Be careful with loop conditions — check when they become false
- **Show condition results:** Include a column for the result of conditions if it helps
- **Record outputs:** Add an output column to show what the algorithm produces
- **Cross out old values:** Some people prefer to cross out old values; others just add new rows

Worked Example 11 – Completing a trace table

Question: Complete the trace table for the following algorithm when the input is 4:

```
INPUT num result ← 1 FOR i ← 1 TO num result ← result * i ENDFOR
OUTPUT result
```

Model answer:

num	result	i	Output
4	1		
	1	1	
	2	2	
	6	3	
	24	4	24

The algorithm outputs 24. It calculates the factorial of the input (4! = 4 × 3 × 2 × 1 = 24).

Exam tip: Work through each iteration carefully. In a FOR loop, track the loop counter (i) as well as other variables.

Worked Example 12 – Determining algorithm purpose

Question: Use a trace table to determine what the following algorithm does. Show your trace table for the input [3, 7, 2, 9, 4].

```
INPUT numbers (a list) max ← numbers[0] FOR i ← 1 TO LENGTH(numbers)
- 1 IF numbers[i] > max THEN max ← numbers[i] ENDIF ENDFOR OUTPUT max
```

Model answer:

i	numbers[i]	max	numbers[i] > max?
-	-	3	-
1	7	7	True
2	2	7	False
3	9	9	True
4	4	9	False

Output: 9
Purpose: The algorithm finds the largest/maximum value in a list.

Exam tip: When determining purpose, describe WHAT the algorithm does, not HOW it does it. Use clear language like "finds the largest value" rather than "loops through and compares".

Extended response practice. AQA exams include extended response questions on computational thinking. Here is an example.

Extended Response – Applying computational thinking
Question: A school wants to create a system for students to borrow books from the library.
Explain how a programmer could use decomposition and abstraction to design this system. [6 marks]

Model answer (Level 3, 5—6 marks):
Decomposition would be used to break the library system into smaller, manageable sub-problems. These might include:

- Recording which books are available in the library
- Recording student information
- Processing book loans (checking out a book)
- Processing book returns
- Tracking overdue books and fines
- Searching for books by title or author

Each sub-problem can be worked on separately. For example, the "processing book loans" sub-problem could be further decomposed into: checking if the book is available, recording who borrowed it, and updating the due date.

Abstraction would be used to remove unnecessary detail and focus on what the system needs. For example:

- A book might be represented by just its title, author, ISBN, and availability status — not its physical dimensions, weight, or cover colour
- A student might be represented by their student ID, name, and list of borrowed books — not their home address or timetable

This simplifies the system by only storing information that is relevant to the borrowing process, making it easier to design and implement.

Exam tip (6-mark questions): For full marks, you must: explain BOTH decomposition AND abstraction, give specific examples relevant to the scenario (library/books/students), and explain WHY each technique is useful. Generic explanations without context will only reach Level 1—2.

Worked examples summary

This page contains 12 worked examples plus one extended response, covering all the key skills for computational thinking. Use this table to find specific types of questions for revision.

Example	Topic	Question type	Marks
1	Defining algorithm	Define	1
2	Algorithm vs program	Explain	2
3	Inputs, processing, outputs	Identify	3
4	Algorithm efficiency	Compare	2
5	Defining decomposition	Define	2
6	Applying decomposition	Apply	4
7	Defining abstraction	Define	1
8	Applying abstraction	Explain	2
9	Writing pseudo-code	Write algorithm	4
10	Writing Python code	Write program	4
11	Completing trace table	Complete table	4
12	Determining algorithm purpose	Trace and explain	4
Ext.	Decomposition and abstraction in context	Explain (extended response)	6

Quick reference

Key definitions (mark-scheme style)

Term	Definition
Algorithm	A sequence of steps that can be followed to complete a task.
Decomposition	Breaking a problem into a number of sub-problems, so that each sub-problem accomplishes an identifiable task.
Abstraction	The process of removing unnecessary detail from a problem.
Pseudo-code	A structured, language-independent way of writing algorithms using plain English and programming-style keywords.
Flowchart	A visual representation of an algorithm using standard symbols connected by arrows.
Trace table	A table used to trace the values of variables as an algorithm is executed.

Flowchart symbols summary

Symbol	Shape	Use
Terminal	Oval	Start / End
Process	Rectangle	Calculation or action
Decision	Diamond	Yes/No question
Input/Output	Parallelogram	Data in/out
Arrow	Line with arrowhead	Flow direction

AQA pseudo-code reference

Assignment and I/O

- `variable ← value`
- `INPUT variable`
- `OUTPUT value`

Selection and iteration

- `IF...THEN...ELSE...ENDIF`
- `FOR...TO...ENDFOR`
- `WHILE...ENDWHILE`
- `REPEAT...UNTIL`

Typical pitfalls

Pitfall 1 — Confusing algorithm and program.
Wrong: "An algorithm is a program."
Right: An algorithm is the PLAN. A program is the IMPLEMENTATION of an algorithm in a programming language.

Pitfall 2 — Incomplete decomposition definition.
Wrong: "Breaking a problem into smaller parts."
Right: Breaking a problem into sub-problems so that each sub-problem accomplishes an identifiable task.

Pitfall 3 — Confusing decomposition and abstraction.
Wrong: Using the terms interchangeably.
Right: Decomposition = breaking into parts. Abstraction = removing unnecessary detail.

Pitfall 4 — Missing loop iterations in trace tables.
Wrong: Skipping iterations or miscounting.
Right: Work through EVERY iteration. Check loop conditions carefully — when do they become false?

Pitfall 5 — Wrong flowchart symbols.
Wrong: Using rectangles for decisions.
Right: Decisions = diamond. Processes = rectangle. I/O = parallelogram. Start/End = oval.

Pitfall 6 — Vague algorithm purpose descriptions.
Wrong: "It loops through and does stuff."
Right: Describe WHAT it achieves: "finds the largest value", "calculates the average", "counts occurrences".

Pitfall 7 — Incorrect boundary conditions.
Wrong: `IF x > 10` when it should be `IF x >= 10`
Right: Check the question carefully. "10 or more" needs >=, "more than 10" needs >.

Pitfall 8 — Forgetting to initialise variables.
Wrong: Using total before setting it to 0.

Right: Always initialise counters and accumulators before using them (e.g., `total ← 0`).

Pitfall 9 — Generic abstraction examples.
Wrong: "Remove unnecessary detail."
Right: Give specific examples: WHAT is kept, WHAT is removed, and WHY it's unnecessary for this purpose.

Pitfall 10 — Not answering in required format.
Wrong: Writing flowchart when asked for pseudo-code.
Right: Check the question — it will state the required format. Follow it exactly.

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.